

WEEK-3

TASK 1 – DOCUMENT MODELLING

STEP 1:- Start MongoDB

```
[test> use bookflow_db
switched to db bookflow_db
[bookflow_db> db.createCollection("book_metadata")
{ ok: 1 }
[bookflow_db> show collections
book_metadata
bookflow_db> ]
```

Figure 1: Creating the book_metadata collection in the BookFlow MongoDB database

STEP 2 & 3:- INSERT THE FIRST BOOK DOCUMENT

```
bookflow_db> db.book_metadata.find({ title: "The Silent Algorithm" })
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    author: 'R. Menon',
    genre: 'Technology',
    publishedYear: 2022,
    formats: [ 'Hardcover', 'Digital', 'Audiobook' ],
    specs: {
      pages: 340,
      fileSize_MB: 4.2,
      drm: true,
      narrator: 'K. Rao',
      runtime_min: 610
    },
    avgRating: 4.6,
    reviews: [
      {
        member_id: 'M1001',
        rating: 5,
        comment: 'Excellent deep dive into digital systems design.',
        date: ISODate('2023-01-15T00:00:00.000Z')
      },
      {
        member_id: 'M1042',
        rating: 4,
        comment: 'Good but the Digital edition had formatting issues.',
        date: ISODate('2023-02-20T00:00:00.000Z')
      }
    ]
  }
]
bookflow_db> ]
```

Figure 2: Inserting and displaying The Silent Algorithm document with embedded reviews and referenced member IDs

STEP 4 & 5:- INSERT THE REMAINING 3 BOOKS

```
[bookflow_db> db.book_metadata.find({}, { title: 1, _id: 0 })
[
  { title: 'The Silent Algorithm' },
  { title: 'Ocean of Code' },
  { title: 'Whispers of Yesterday' },
  { title: 'Quantum Horizons' }
]
bookflow_db> █
```

Figure 3: Displaying all book documents inserted into the book_metadata collection

TASK 2 – ADVANCED QUERYING

STEP 1:- Books rated above 4

```
bookflow_db> db.book_metadata.find(
|   { avgRating: { $gt: 4 } },
|   { title: 1, avgRating: 1, _id: 0 }
| );
[
  { title: 'The Silent Algorithm', avgRating: 4.6 },
  { title: 'Ocean of Code', avgRating: 4.2 },
  { title: 'Quantum Horizons', avgRating: 4.9 }
]
bookflow_db> █
```

Figure 4: Displaying books with average rating greater than 4 using the \$gt operator

STEP 2:- BOOKS AVAILABLE IN SPECIFIC FORMATS USING \$in

```
bookflow_db> db.book_metadata.find(
|   { formats: { $in: ["Digital", "Audiobook"] } },
|   { title: 1, formats: 1, _id: 0 }
| );
[
  {
    title: 'The Silent Algorithm',
    formats: [ 'Hardcover', 'Digital', 'Audiobook' ]
  },
  { title: 'Ocean of Code', formats: [ 'Digital', 'Paperback' ] },
  { title: 'Quantum Horizons', formats: [ 'Digital', 'Audiobook' ] }
]
█
```

Figure 5: Displaying books available in Digital or Audiobook formats using the \$in operator

STEP 3:- COMBINE \$gt AND \$in

```
bookflow_db> db.book_metadata.find({
|   avgRating: { $gt: 4 },
|   formats: { $in: ["Digital"] }
| });
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    author: 'R. Menon',
    genre: 'Technology',
    publishedYear: 2022,
    formats: [ 'Hardcover', 'Digital', 'Audiobook' ],
    specs: {
      pages: 340,
      fileSize_MB: 4.2,
      drm: true,
      narrator: 'K. Rao',
      runtime_min: 610
    },
    avgRating: 4.6,
    reviews: [
      {
        member_id: 'M1001',
        rating: 5,
        comment: 'Excellent deep dive into digital systems design.',
        date: ISODate('2023-01-15T00:00:00.000Z')
      },
      {
        member_id: 'M1042',
        rating: 4,
        comment: 'Good but the Digital edition had formatting issues.',
        date: ISODate('2023-02-20T00:00:00.000Z')
      }
    ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b74'),
    title: 'Ocean of Code',
    author: 'A. Fernandes',
    genre: 'Technology',
    publishedYear: 2021,
    formats: [ 'Digital', 'Paperback' ],
    specs: { pages: 210, fileSize_MB: 2.1, drm: false },
    avgRating: 4.2,
    reviews: [
      {
        member_id: 'M2001',
        rating: 4,
        comment: 'Solid Digital read, clean layout.',
        date: ISODate('2022-05-10T00:00:00.000Z')
      },
      {
        member_id: 'M2002',
        rating: 5,
        comment: 'Best programming book this year.',
        date: ISODate('2022-06-01T00:00:00.000Z')
      }
    ]
  }
],
```

```

  {
    _id: ObjectId('6a84708166d584edd9655b76'),
    title: 'Quantum Horizons',
    author: 'R. Menon',
    genre: 'Science',
    publishedYear: 2023,
    formats: [ 'Digital', 'Audiobook' ],
    specs: { fileSize_MB: 5.6, drm: true, narrator: 'P. Iyer', runtime_min: 540 },
    avgRating: 4.9,
    reviews: [
      {
        member_id: 'M4001',
        rating: 5,
        comment: 'The Digital format made annotations so easy.',
        date: ISODate('2023-08-01T00:00:00.000Z')
      },
      {
        member_id: 'M4002',
        rating: 5,
        comment: 'Mind-bending and well narrated.',
        date: ISODate('2023-09-14T00:00:00.000Z')
      }
    ]
  }
]
]
```

Figure 6: Displaying books with rating greater than 4 and Digital format using combined query conditions

STEP 4:- REGEX QUERY: BOOKS WITH "Digital" FORMAT

```
bookflow_db> db.book_metadata.find(
|   { formats: { $regex: "Digital", $options: "i" } },
|   { title: 1, formats: 1 }
| );
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    formats: [ 'Hardcover', 'Digital', 'Audiobook' ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b74'),
    title: 'Ocean of Code',
    formats: [ 'Digital', 'Paperback' ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b76'),
    title: 'Quantum Horizons',
    formats: [ 'Digital', 'Audiobook' ]
  }
]
bookflow_db>
```

Figure 7: Displaying books with Digital format using the \$regex operator

STEP 5:- REGEX QUERY ON REVIEW COMMENTS

```
|   { title: 1, formats: 1 }
| );
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    formats: [ 'Hardcover', 'Digital', 'Audiobook' ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b74'),
    title: 'Ocean of Code',
    formats: [ 'Digital', 'Paperback' ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b76'),
    title: 'Quantum Horizons',
    formats: [ 'Digital', 'Audiobook' ]
  }
]
bookflow_db> db.book_metadata.find(
|   { "reviews.comment": { $regex: "Digital", $options: "i" } },
|   { title: 1, "reviews.$": 1 }
| );
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    reviews: [
      {
        member_id: 'M1001',
        rating: 5,
        comment: 'Excellent deep dive into digital systems design.',
        date: ISODate('2023-01-15T00:00:00.000Z')
      }
    ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b74'),
    title: 'Ocean of Code',
    reviews: [
      {
        member_id: 'M2001',
        rating: 4,
        comment: 'Solid Digital read, clean layout.',
        date: ISODate('2022-05-10T00:00:00.000Z')
      }
    ]
  },
  {
    _id: ObjectId('6a84708166d584edd9655b76'),
    title: 'Quantum Horizons',
    reviews: [
      {
        member_id: 'M4001',
        rating: 5,
        comment: 'The Digital format made annotations so easy.',
        date: ISODate('2023-08-01T00:00:00.000Z')
      }
    ]
  }
]
```

Figure 8: Searching embedded review comments containing the word Digital using the \$regex operator

TASK 3 — AGGREGATION PIPELINE: “TOP RATED” REPORT

STEP 1:- Run the Aggregation Pipeline

```
bookflow_db> db.book_metadata.aggregate([
  { $match: { publishedYear: { $gt: 2020 } } },
  { $unwind: "$reviews" },
  {
    $group: {
      _id: "$title",
      avgReviewRating: { $avg: "$reviews.rating" },
      totalReviews: { $sum: 1 },
      author: { $first: "$author" }
    }
  },
  { $sort: { avgReviewRating: -1 } },
  {
    $project: {
      _id: 0,
      title: "$_id",
      author: 1,
      avgReviewRating: { $round: ["$avgReviewRating", 2] },
      totalReviews: 1
    }
  }
]);
[
  {
    totalReviews: 2,
    author: 'R. Menon',
    title: 'Quantum Horizons',
    avgReviewRating: 5
  },
  {
    totalReviews: 2,
    author: 'A. Fernandes',
    title: 'Ocean of Code',
    avgReviewRating: 4.5
  },
  {
    totalReviews: 2,
    author: 'R. Menon',
    title: 'The Silent Algorithm',
    avgReviewRating: 4.5
  }
]
```

Figure 9: Generating the Top Rated books report using the MongoDB aggregation pipeline

TASK 4 – INDEXING FOR SPEED

STEP 1:- CREATE TEXT INDEX ON REVIEW COMMENTS

```
[bookflow_db> db.book_metadata.createIndex({ "reviews.comment": "text" });
reviews.comment_text
[bookflow_db> db.book_metadata.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  {
    v: 2,
    key: { _fts: 'text', _ftsx: 1 },
    name: 'reviews.comment_text',
    weights: { 'reviews.comment': 1 },
    default_language: 'english',
    language_override: 'language',
    textIndexVersion: 3
  }
]
bookflow_db> █
```

Figure 10: Creating a text index on the reviews.comment field in the book_metadata collection

STEP 2:- QUERY USING THE TEXT INDEX

```
bookflow_db> db.book_metadata.find(
|   { $text: { $search: "Digital" } },
|   { title: 1, score: { $meta: "textScore" } }
| ).sort({ score: { $meta: "textScore" } });
[
  {
    _id: ObjectId('6a846f9b66d584edd9655b73'),
    title: 'The Silent Algorithm',
    score: 1.1833333333333333
  },
  {
    _id: ObjectId('6a84708166d584edd9655b74'),
    title: 'Ocean of Code',
    score: 0.6
  },
  {
    _id: ObjectId('6a84708166d584edd9655b76'),
    title: 'Quantum Horizons',
    score: 0.6
  }
]
bookflow_db> █
```

Figure 11: Searching review comments for the word Digital using the MongoDB text index

STEP 3:- VERIFY TEXT INDEX USING .explain()

```
bookflow_db> db.book_metadata.find(
|   { $text: { $search: "Digital" } }
| ) .explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'bookflow_db.book_metadata',
    parsedQuery: {
      '$text': {
        '$search': 'Digital',
        '$language': 'english',
        '$caseSensitive': false,
        '$diacriticSensitive': false
      }
    },
    indexFilterSet: false,
    queryHash: '4FB85230',
    planCacheShapeHash: '4FB85230',
    planCacheKey: 'F34F3876',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'TEXT_MATCH',
      nss: 'bookflow_db.book_metadata',
      indexPrefix: {},
      indexName: 'reviews.comment_text',
      parsedTextQuery: {
        terms: [ 'digit' ],
        negatedTerms: [],
        phrases: [],
        negatedPhrases: []
      },
      textIndexVersion: 3,
      inputStage: {
        stage: 'FETCH',
        nss: 'bookflow_db.book_metadata',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'bookflow_db.book_metadata',
          keyPattern: { _fts: 'text', _ftsx: 1 },
          indexName: 'reviews.comment_text',
          isMultiKey: true,
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: {}
        }
      }
    },
    rejectedPlans: []
  },
}
```

Figure 12: Verifying the usage of the text index using explain with executionStats

STEP 4:- COMPARE WITH NON-INDEXED REGEX SCAN

```
executionStats: {
  executionSuccess: true,
  nReturned: 3,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'COLLSCAN',
    filter: { 'reviews.comment': { '$regex': 'Digital', '$options': 'i' } },
    nReturned: 3,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 3,
    needTime: 1,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'bookflow_db.book_metadata',
    direction: 'forward',
    docsExamined: 4
  }
}
```

Figure 13: Comparing a non-indexed regex search using explain and showing the COLLSCAN execution stage

STEP 5:- CREATE SUPPORTING INDEXES

```
[bookflow_db> db.book_metadata.createIndex({ formats: 1 });
formats_1
[bookflow_db> db.book_metadata.createIndex({ avgRating: -1 });
avgRating_-1
```

```
inputStage: {
  stage: 'IXSCAN',
  nReturned: 3,
  executionTimeMillisEstimate: 0,
  works: 4,
  advanced: 3,
  needTime: 0,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
  isEOF: 1,
  nss: 'bookflow_db.book_metadata',
  keyPattern: { avgRating: -1 },
  indexName: 'avgRating_-1',
  isMultiKey: false,
  multiKeyPaths: { avgRating: [] },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: { avgRating: [ '[inf, 4)' ] },
  keysExamined: 3,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0,
  peakTrackedMemBytes: 0
}
```

Figure 14 & 15: Creating supporting indexes on formats and avgRating fields and Verifying the avgRating index using explain with executionStats